

UnifAI / Raksha Enterprise Docker & SSL/TLS Configuration Guide

Audience: DevOps Engineers, Cloud Engineers, SREs, Systems & Security Administrators

Scope: Docker topology, multi-container orchestration, exhaustive Docker command handbook, SSL/TLS certificate acquisition via Certbot, OpenSSL self-signed generation, Browser Guard Root CA generation, OS trust store deployment commands, Nginx reverse proxy integration, and diagnostics.

Baseline: Repo commit **16d4c3e**

1. Container Topology & Architecture

UnifAI deploys as a multi-container Docker Compose application:

[illegible]

Services Defined in `docker-compose.yml`

1. raksha_tech (Primary Application):

- Multi-stage build (**deploy/docker/Dockerfile.local**): **node:25-alpine** builds React UI $\rightarrow$ **golang:1.26.4-alpine** builds Go binary $\rightarrow$ **alpine:3.23** lightweight runtime.
- Serves the UI, **/v1/*** inference gateway, **/api/*** management plane, and Browser AI endpoints.

2. raksha_browser_ai_proxy (Optional Network Proxy):

- Enabled via profile: **docker compose --profile network-proxy up -d**.
- Runs **mitmproxy/mitmproxy:latest** with the Browser Guard Python addon for office/lab PAC proxying.

2. Complete Docker & Docker Compose Command Handbook

All commands should be executed from the project root (**d:\unifai_project** or **/opt/unifai**).

2.1 Network Setup (Prerequisite)

The compose file references an external network for Ollama (**1panel-network**). Create it once:

```
# Create external Ollama network (required before startup) docker network create 1panel-network # Create
primary application bridge network (if custom name specified in .env) docker network create raksha-network #
List all Docker networks docker network ls
```

2.2 Container Startup & Rebuild Commands

```
# 1. Start primary UnifAI container in background (detached mode) docker compose up -d # 2. Start both
primary backend AND the optional network proxy docker compose --profile network-proxy up -d # 3. Start with
fresh rebuild (forces re-compilation of UI and Go binary) docker compose up -d --build # 4. Clean rebuild
without using any Docker build cache docker compose build --no-cache && docker compose up -d # 5. Start in
foreground (prints logs directly to console; Ctrl+C to stop) docker compose up
```

2.3 Status, Monitoring & Inspection Commands

```
# View status of running containers, ports, and healthcheck status docker compose ps # View status of all
containers including stopped or exited ones docker compose ps -a # View real-time CPU %, Memory usage,
Network I/O, and PIDs docker stats raksha_tech # Inspect detailed container JSON metadata (IP address,
mounts, environment) docker inspect raksha_tech # Inspect network to see assigned IP addresses of connected
containers docker network inspect raksha-network docker network inspect 1panel-network
```

2.4 Live Logs & Diagnostics Commands

```
# Follow real-time logs of all containers docker compose logs -f # Follow logs of primary Go backend with
last 100 lines docker compose logs -f --tail=100 raksha_tech # Follow logs of network proxy container docker
compose logs -f --tail=100 raksha_browser_ai_proxy # Filter logs for errors or warnings docker compose logs
raksha_tech | grep -i "error" # Check startup banner and plugin status table docker compose logs raksha_tech
| grep -E "successfully started|plugin status:"
```

2.5 Container Execution & In-Container Debugging

```
# Open an interactive shell inside the running backend container docker exec -it raksha_tech /bin/sh # Test
backend healthcheck directly from inside the container docker exec -it raksha_tech curl -v
http://localhost:6000/health # Verify environment variables loaded inside the container docker exec -it
raksha_tech env | grep -E "APP_PORT|DB_HOST|SERVER_DOMAIN" # Inspect data directory contents inside container
docker exec -it raksha_tech ls -la /app/data
```

2.6 Stopping, Restarting & Cleanup Commands

```
# Restart the backend container gracefully (after modifying .env) docker compose restart raksha_tech # Stop
running containers without deleting them docker compose stop # Stop and remove containers, networks, and
internal links docker compose down # Stop and remove containers, networks, and anonymous volumes (bind mounts
preserved) docker compose down -v # Remove dangling unused images docker image prune -f # Clean up stopped
containers, unused networks, and dangling images docker system prune -f # Full nuclear prune: removes all
unused images, stopped containers, and unused volumes (CAUTION) docker system prune -a --volumes -f
```

3. SSL/TLS Certificate Generation & Management

Production enterprise deployments require valid TLS certificates. This section provides exact commands for obtaining certificates via Let's Encrypt (Certbot), generating private self-signed certificates with OpenSSL, and generating the Root CA certificate for Browser Guard interception.

3.1 Obtaining Free Public SSL Certificates with Let's Encrypt (Certbot)

Certbot provides trusted certificates accepted by all modern operating systems and web browsers.

Prerequisites

- Your domain name (e.g. **unifai.yourcompany.com**) must have an active DNS **A** record pointing to your server's public IP address.
- Ports **80** and **443** must be open on your server firewall.

Method A: Standalone Mode (Before starting Nginx)

```
# Install Certbot sudo apt update && sudo apt install -y certbot # Request certificate in standalone mode
sudo certbot certonly --standalone \ --preferred-challenges http \ -d unifai.yourcompany.com \ --email
security-admin@yourcompany.com \ --agree-tos \ --no-eff-email # Certificates are saved to: # Certificate &
Full Chain: /etc/letsencrypt/live/unifai.yourcompany.com/fullchain.pem # Private Key:
/etc/letsencrypt/live/unifai.yourcompany.com/privkey.pem
```

Method B: Webroot Mode (With Nginx already running)

```
# Create certbot challenge directory sudo mkdir -p /var/www/certbot # Request certificate using webroot sudo
certbot certonly --webroot \ -w /var/www/certbot \ -d unifai.yourcompany.com \ --email
security-admin@yourcompany.com \ --agree-tos # Test automated certificate renewal sudo certbot renew
--dry-run
```

Method C: Automated Renewal Crontab

Certbot certificates expire after 90 days. Set up automatic renewal:

```
# Add cron job to renew twice daily and reload Nginx echo "0 3,15 * * * root certbot renew --quiet
--post-hook 'systemctl reload nginx'" | sudo tee -a /etc/crontab
```

3.2 Generating Self-Signed SSL/TLS Certificates with OpenSSL

For internal labs, air-gapped VPCs, or testing environments without public DNS:

Single-Command 4096-bit RSA Certificate with SAN (Subject Alternative Names)

```
# Create directory for custom certificates sudo mkdir -p /etc/ssl/unifai && cd /etc/ssl/unifai # Generate
private key and self-signed certificate valid for 3 years (1095 days) sudo openssl req -x509 -nodes -days
1095 -newkey rsa:4096 \ -keyout unifai_server.key \ -out unifai_server.crt \ -subj
"/C=US/ST=California/L=SanFrancisco/O=YourCompany/OU=IT/CN=unifai.yourcompany.com" \ -addext
"subjectAltName=DNS:unifai.yourcompany.com,DNS:localhost,IP:127.0.0.1" # Restrict private key permissions
sudo chmod 600 unifai_server.key sudo chmod 644 unifai_server.crt
```

Inspecting and Verifying the Generated Certificate

```
# Print certificate human-readable details, validity dates, and SAN extensions openssl x509 -in
unifai_server.crt -text -noout # Verify certificate matches private key modulus openssl x509 -noout -modulus
-in unifai_server.crt | openssl md5 openssl rsa -noout -modulus -in unifai_server.key | openssl md5 # Output
MD5 hashes MUST be identical!
```

3.3 Generating Root CA Certificate for Raksha Browser Guard (MITM Proxy)

To inspect HTTPS traffic between employee browsers and target Generative AI websites (ChatGPT, Claude, Gemini), Browser Guard requires an internal Root Certificate Authority (CA).

Step-by-Step Root CA Creation with OpenSSL

```
# Create CA authority directory mkdir -p /opt/unifai/certs/ca && cd /opt/unifai/certs/ca # 1. Generate Root CA Private Key (4096-bit RSA) openssl genrsa -out raksha-ca.key 4096 chmod 400 raksha-ca.key # 2. Generate Root CA Certificate (valid for 10 years / 3650 days) openssl req -x509 -new -nodes -key raksha-ca.key -sha256 -days 3650 \ -out raksha-ca.crt \ -subj "/C=US/ST=State/L=City/O=Raksha Security Enterprise/OU=Fleet Security/CN=Raksha Root CA - Enterprise AI Guard" # 3. Create combined PEM file for mitmproxy cat raksha-ca.key raksha-ca.crt > mitmproxy-ca.pem chmod 600 mitmproxy-ca.pem
```

3.4 Installing the Root CA Certificate into Client Operating Systems

For Browser Guard to inspect HTTPS sessions without browser security warnings (**ERR_CERT_AUTHORITY_INVALID**), the Root CA certificate (**raksha-ca.crt**) must be installed into the client trust store.

A. Windows Client Installation (Automated via CMD / PowerShell / Inno Setup)

Execute as **Administrator**:

```
:: Using built-in Windows certutil utility certutil -addstore -f "Root" "C:\path\to\raksha-ca.crt"
```

Verify installation in Windows Certificate Manager:

```
certutil -verifystore "Root" "Raksha Root CA - Enterprise AI Guard"
```

B. macOS Client Installation (Terminal / Jamf / Kandji)

Execute with **sudo**:

```
# Add certificate to macOS System Keychain and grant full trust for SSL/TLS sudo security add-trusted-cert \ -d \ -r trustRoot \ -k /Library/Keychains/System.keychain \ /path/to/raksha-ca.crt
```

Verify on macOS:

```
security find-certificate -c "Raksha Root CA - Enterprise AI Guard" /Library/Keychains/System.keychain
```

C. Linux (Ubuntu / Debian) Client Installation

```
# Copy certificate to trusted store directory sudo cp raksha-ca.crt /usr/local/share/ca-certificates/raksha-ca.crt # Update system certificate authorities sudo update-ca-certificates
```

D. Mozilla Firefox NSS Database Installation (Cross-Platform)

Firefox uses its own internal NSS certificate database rather than the OS trust store. To install into Firefox profiles via CLI:

```
# Install libnss3-tools sudo apt install -y libnss3-tools # Linux # or choco install nss -y # Windows # Import into all Firefox user profiles for certDB in $(find $HOME/.mozilla/firefox* -name "cert9.db"); do certdir=$(dirname ${certDB}) certutil -A -n "Raksha Root CA" -t "TCu,Cu,Tu" -i /path/to/raksha-ca.crt -d sql:${certdir} done
```

4. Reverse Proxy Integration: Connecting Nginx to Docker

When deploying via Docker Compose, Nginx runs on the host server (or in a dedicated edge container) and proxies incoming HTTPS requests to the Docker container port (**6000**).

4.1 Production Nginx Configuration File

Save to **/etc/nginx/sites-available/unifai:**

```
# Upstream pointing to Docker published port on localhost upstream docker_unifai_backend { server
127.0.0.1:6000; keepalive 32; } # Plain HTTP to HTTPS redirect server { listen 80; listen [::]:80;
server_name unifai.yourcompany.com; return 301 https://$host$request_uri; } # Production HTTPS Server server
{ listen 443 ssl http2; listen [::]:443 ssl http2; server_name unifai.yourcompany.com; # SSL Certificates
ssl_certificate /etc/letsencrypt/live/unifai.yourcompany.com/fullchain.pem; ssl_certificate_key
/etc/letsencrypt/live/unifai.yourcompany.com/privkey.pem; # Protocols and Ciphers ssl_protocols TLSv1.2
TLSv1.3; ssl_ciphers HIGH:!aNULL:!MD5; ssl_prefer_server_ciphers on; ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d; # Max upload limit for Guard installers and document embeddings client_max_body_size
100M; # Proxy to Docker container location / { proxy_pass http://docker_unifai_backend; proxy_http_version
1.1; # WebSocket support proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade"; #
Request forwarding headers proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto https;
proxy_set_header X-Forwarded-Host $host; proxy_set_header X-Forwarded-Port 443; # Disable buffering for
real-time streaming AI responses (SSE) proxy_buffering off; proxy_read_timeout 600s; proxy_connect_timeout
60s; } }
```

Enable and reload Nginx:

```
sudo ln -sf /etc/nginx/sites-available/unifai /etc/nginx/sites-enabled/ sudo nginx -t && sudo systemctl
reload nginx
```

5. SSL/TLS Verification & Diagnostics Commands

Always test your SSL configuration and verify end-to-end handshake:

```
# 1. Test TLS Handshake and inspect presented certificate chain openssl s_client -connect
unifai.yourcompany.com:443 -servername unifai.yourcompany.com # 2. Check HTTP status, redirection, and
headers using curl curl -vI https://unifai.yourcompany.com/health # 3. Verify TLS 1.3 protocol negotiation
openssl s_client -connect unifai.yourcompany.com:443 -tls1_3 # 4. Check certificate expiration date directly
from remote server echo | openssl s_client -connect unifai.yourcompany.com:443 -servername
unifai.yourcompany.com 2>/dev/null | openssl x509 -noout -dates
```

End of Docker & SSL/TLS Configuration Guide.